

Festival

Nayra sedang bermain sebuah permainan di festival dengan hadiah utama berupa tamasya ke Laguna Colorada. Permainan ini terdiri dari penggunaan token untuk melakukan pembelian kupon. Pembelian sebuah kupon bisa jadi menambah jumlah token. Tujuan dari permainan ini adalah untuk mendapatkan kupon sebanyak mungkin.

Nayra memulai permainan dengan A token. Terdapat N kupon yang diberi nomor dari 0 hingga $N - 1$. Nayra harus membayar $P[i]$ token ($0 \leq i < N$) untuk membeli kupon i (dan ia harus memiliki setidaknya $P[i]$ token sebelum melakukan pembelian). Nayra dapat membeli setiap kupon paling banyak satu kali.

Selain itu, setiap kupon i ($0 \leq i < N$) memiliki sebuah **tipe** yang ditandai dengan $T[i]$, sebuah bilangan bulat **di antara 1 dan 4 inklusif**. Setelah Nayra membeli kupon i , sisa token yang ia miliki akan dikalikan dengan $T[i]$. Lebih tepatnya, jika Nayra memiliki X token pada suatu saat di dalam permainan dan membeli kupon i (yang mengharuskan $X \geq P[i]$), maka ia akan memiliki $(X - P[i]) \cdot T[i]$ token setelah pembelian tersebut.

Tugas Anda adalah menentukan kupon-kupon yang harus dibeli beserta urutan pembeliannya, untuk memaksimalkan banyaknya **kupon** yang dimiliki Nayra pada akhirnya. Jika terdapat lebih dari satu urutan pembelian yang memaksimalkan banyak kupon, Anda dapat menjawab dengan urutan manapun.

Detail Implementasi

Anda harus mengimplementasikan prosedur berikut.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : banyaknya token yang dimiliki oleh Nayra pada awalnya.
- P : *array* dengan panjang N yang merupakan harga dari kupon.
- T : *array* dengan panjang N yang merupakan tipe dari kupon.
- Prosedur ini dipanggil tepat sekali untuk setiap kasus uji.

Prosedur ini harus mengembalikan sebuah *array* R yang merupakan pembelian Nayra dengan syarat sebagai berikut:

- Panjang R adalah banyaknya kupon maksimal yang dapat dibeli oleh Nayra.
- Elemen dari *array* adalah indeks kupon yang harus dibeli oleh Nayra secara kronologis. Lebih tepatnya, ia membeli kupon $R[0]$, lalu kupon $R[1]$, dan seterusnya.
- Semua elemen R harus unik.

Jika tidak ada kupon yang dapat dibeli, R harus merupakan *array* kosong.

Batasan

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ untuk setiap i sehingga $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ untuk setiap i sehingga $0 \leq i < N$.

Subsoal

Subsoal	Nilai	Batasan Tambahan
1	5	$T[i] = 1$ untuk setiap i sehingga $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ untuk setiap i sehingga $0 \leq i < N$.
3	12	$T[i] \leq 2$ untuk setiap i sehingga $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra dapat membeli seluruh N kupon (dengan suatu urutan).
6	16	$(A - P[i]) \cdot T[i] < A$ untuk setiap i sehingga $0 \leq i < N$.
7	18	Tidak ada batasan tambahan.

Contoh

Contoh 1

Perhatikan pemanggilan berikut.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Awalnya, Nayra memiliki $A = 13$ token. Nayra dapat membeli 3 kupon dengan urutan sebagai berikut:

Kupon yang dibeli	Harga kupon	Tipe kupon	Banyak token setelah pembelian
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Dalam contoh ini, tidak mungkin bagi Nayra untuk membeli lebih dari 3 kupon, dan urutan pembelian di atas merupakan satu-satunya cara Nayra dapat membeli 3 kupon. Oleh karena itu, prosedur harus mengembalikan $[2, 3, 0]$.

Contoh 2

Perhatikan pemanggilan berikut.

```
max_coupons(9, [6, 5], [2, 3])
```

Dalam contoh ini, Nayra dapat membeli kedua kupon dengan urutan apapun. Oleh karena itu, prosedur harus mengembalikan $[0, 1]$ atau $[1, 0]$.

Contoh 3

Perhatikan pemanggilan berikut.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Dalam contoh ini, Nayra memiliki satu token, yang tidak cukup untuk membeli kupon apa pun. Oleh karena itu, prosedur harus mengembalikan $[\]$ (*array* kosong).

Contoh Grader

Format masukan:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Format keluaran:

```
S  
R[0] R[1] ... R[S-1]
```

Di sini, S merupakan panjang dari *array* R yang dikembalikan oleh `max_coupons`.